

Integrated Aspects of Asset Management

Algorithmic Trading Case Study: EUR/USD Strategy

Authors:

Tayebbeh Oveisi Fardoye, Sara Eghlidi,
Alejandro Arzola de Leon, Hadi Shahparvari

University of Applied Sciences BFI Vienna

January 25, 2026

Abstract

This report presents the design, evaluation, and deployment of an algorithmic trading strategy for the EUR/USD currency pair using high-frequency 1-minute data. Five systematic trading strategies were evaluated under a strict 80% time-in-market constraint and ranked based on In-Sample Sharpe Ratio optimization. While the selected Momentum strategy exhibited strong In-Sample performance (Sharpe 5.98), its performance deteriorated Out-of-Sample due to a market regime shift. To mitigate this instability, an adaptive strategy-switching framework was implemented, significantly reducing drawdowns. Finally, both strategies were deployed as automated trading systems on a Linux server connected to Interactive Brokers, satisfying all operational and safety requirements.

Contents

1	Introduction	3
2	Data and Methodology	3
2.1	Dataset Description	3
2.2	Time-in-Market Constraint	3
2.3	Performance Metric	4
3	Trading Strategies	4
4	In-Sample Analysis	4
5	Out-of-Sample Performance	5
6	Adaptive Strategy Extension	6
7	Algorithmic Deployment	7
8	Conclusion	7
A	Appendix A: Server Deployment – Base Momentum Strategy	8
B	Appendix B: Server Deployment – Adaptive Strategy Extension	9

1 Introduction

Algorithmic trading plays a central role in modern asset management, particularly in highly liquid financial markets such as foreign exchange. The EUR/USD currency pair is the most actively traded FX instrument globally, offering deep liquidity, low transaction costs, and continuous price discovery, making it a suitable candidate for systematic trading strategies.

A fundamental challenge in quantitative asset management is the non-stationary nature of financial markets. Statistical properties such as volatility, correlation structures, and return distributions evolve over time as market participants and macroeconomic conditions change. As a result, trading strategies optimized for a specific historical regime may fail when market dynamics shift.

In the context of algorithmic trading, data mining refers to the systematic process of testing multiple trading rules and parameter combinations on historical data in order to identify patterns that may generate profitable trading signals. While this approach can uncover useful relationships, it also introduces the risk of overfitting, as strategies optimized on past data may fail to generalize to future market conditions.

The objective of this project is not merely to identify a profitable trading rule, but to demonstrate a complete quantitative asset management workflow, integrating data mining, rigorous validation, and real-world deployment. Particular emphasis is placed on understanding data-mining bias and the importance of Out-of-Sample testing.

2 Data and Methodology

2.1 Dataset Description

The analysis is conducted using 1-minute OHLC data for the EUR/USD currency pair covering the period from November 1, 2024 to October 31, 2025. High-frequency data allows for granular analysis of intraday price movements and realistic modeling of execution dynamics.

The dataset is divided into two non-overlapping periods:

- **In-Sample (Training):** First three months (approximately 60 trading days), used for parameter optimization and strategy selection.
- **Out-of-Sample (Validation):** Remaining nine months, treated as unseen future data.

2.2 Time-in-Market Constraint

A mandatory requirement of the assignment is that trading strategies remain invested in the market for at least 80% of the trading time. Time in market is computed on a minute-by-minute basis as:

$$\text{Time in Market} = \frac{\text{Total Minutes with Non-Zero Position}}{\text{Total Trading Minutes}}$$

All strategies operate under a continuous switching framework, holding either a long (+1) or short (−1) position at all times during trading hours. Consequently, all strategies achieve a 100% time-in-market ratio.

2.3 Performance Metric

Strategy performance is evaluated primarily using the Sharpe Ratio, which measures excess return per unit of risk. Sharpe Ratios are annualized using a scaling factor appropriate for 1-minute data, ensuring comparability across strategies.

3 Trading Strategies

Five rule-based trading strategies were implemented alongside a Buy & Hold benchmark: Momentum, Breakout, RSI Trend, Mean Reversion, and SMA Crossover. Each strategy reflects a distinct economic intuition regarding price formation, ranging from trend persistence to mean reversion dynamics. All strategies are deterministic, fully programmable, and suitable for automated execution.

4 In-Sample Analysis

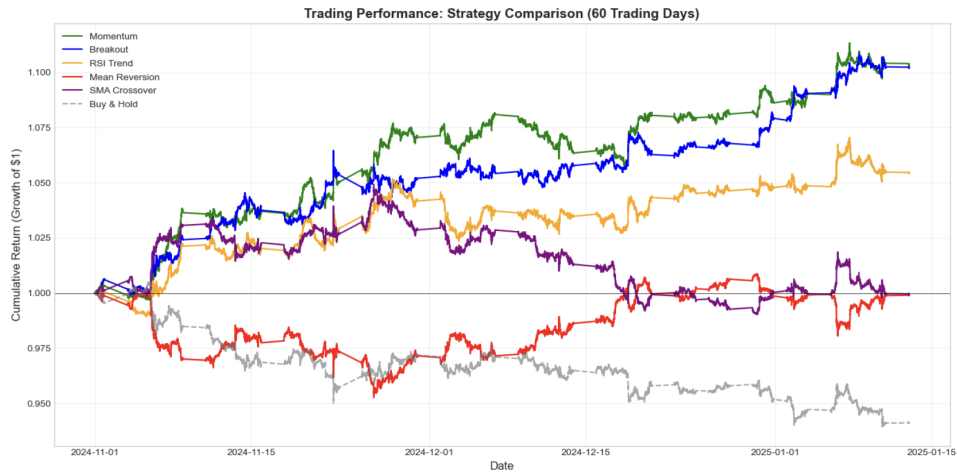


Figure 1: In-Sample cumulative performance of all strategies over the 60-day training window.

Table 1 reports the In-Sample performance metrics for all strategies. The Momentum strategy achieved the highest Sharpe Ratio (5.98) and the largest total return (10.35%), while maintaining a relatively small maximum drawdown of −2.22%.

Table 1: Strategy Ranking – In-Sample (60 Trading Days)

Strategy	Sharpe Ratio	Total Return (%)	Max Drawdown (%)	Time in Market (%)
Momentum	5.98	10.35	−2.22	100
Breakout	5.91	10.22	−2.02	100
RSI Trend	3.25	5.47	−2.68	100
Mean Reversion	0.01	−0.06	−4.86	100
SMA Crossover	−0.01	−0.08	−5.63	100

Based on these results, the Momentum strategy was selected for Out-of-Sample validation.

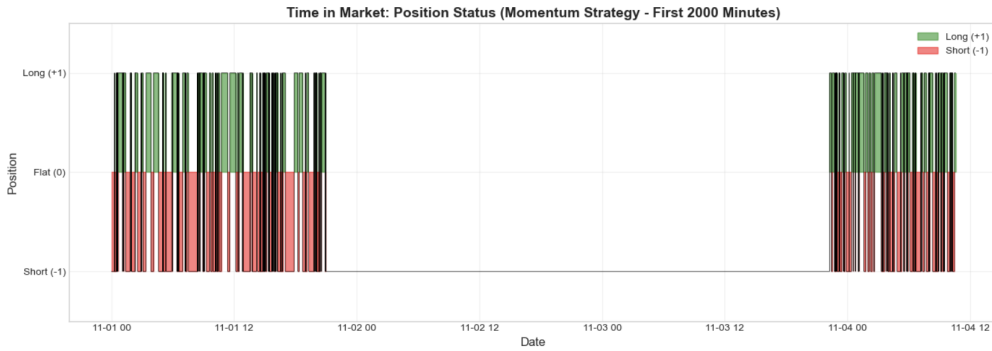


Figure 2: Time-in-Market verification showing continuous long/short exposure.

5 Out-of-Sample Performance

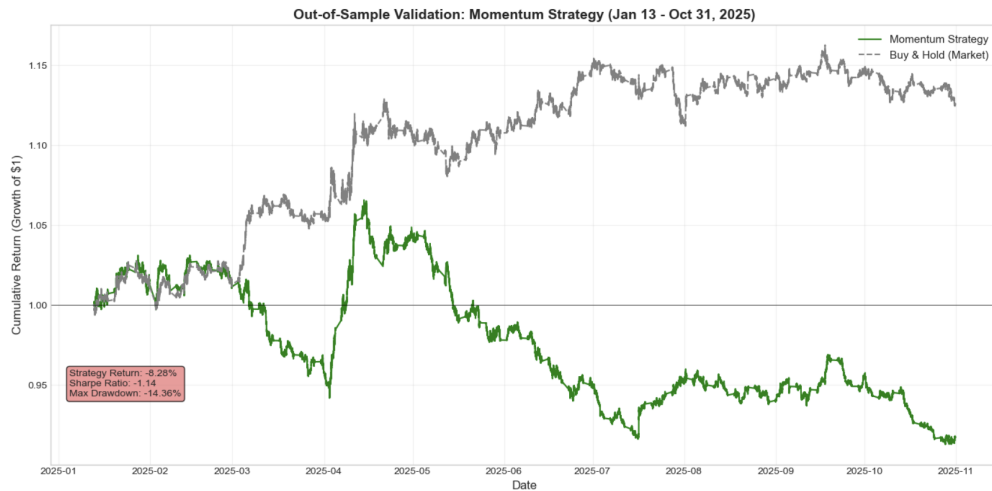


Figure 3: Out-of-Sample performance of the Momentum strategy versus Buy & Hold.

Table 2 summarizes the Out-of-Sample performance of the Momentum strategy. The Sharpe Ratio declined sharply to -1.14 , with a total return of -8.28% and a maximum drawdown of -14.36% .

Table 2: Out-of-Sample Results – Momentum Strategy

Metric	Value
Sharpe Ratio	−1.14
Total Return (%)	−8.28
Annualized Return (%)	−9.96
Max Drawdown (%)	−14.36
Time in Market (%)	100



Figure 4: Out-of-Sample drawdown profile of the Momentum strategy.

Table 3: Comparison of In-Sample and Out-of-Sample Performance

Metric	In-Sample	Out-of-Sample
Sharpe Ratio	5.98	−1.14
Total Return (%)	10.35	−8.28
Max Drawdown (%)	−2.22	−14.36

6 Adaptive Strategy Extension

Table 4 compares the static Momentum strategy with the Adaptive strategy during the Out-of-Sample period.

Table 4: Performance Comparison: Momentum vs Adaptive Strategy

Metric	Momentum Only	Adaptive Strategy
Sharpe Ratio	−1.14	−0.51
Total Return (%)	−8.28	−3.95
Max Drawdown (%)	−14.36	−15.06
Strategy Switches	0	24

The adaptive strategy reduced losses by 4.32 percentage points and improved risk-adjusted performance by dynamically reallocating exposure in response to changing market regimes.

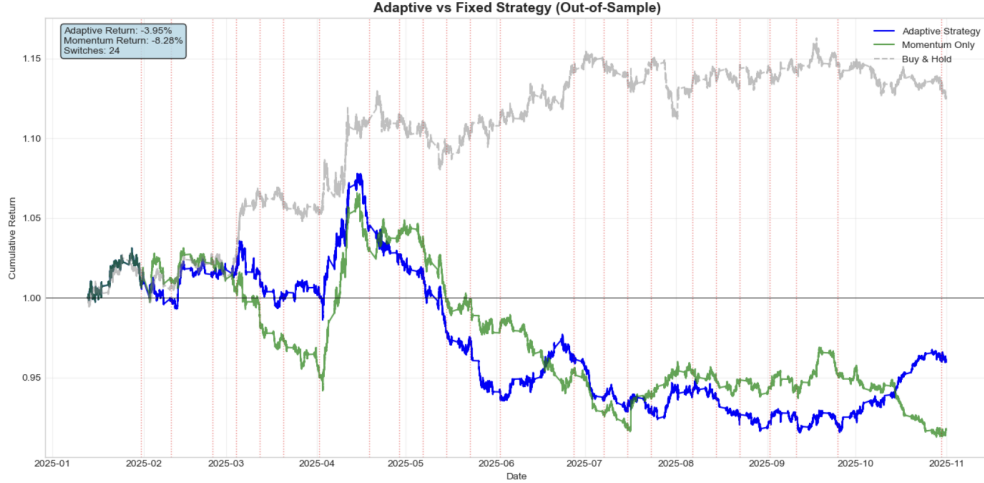


Figure 5: Adaptive strategy versus static Momentum strategy.

7 Algorithmic Deployment

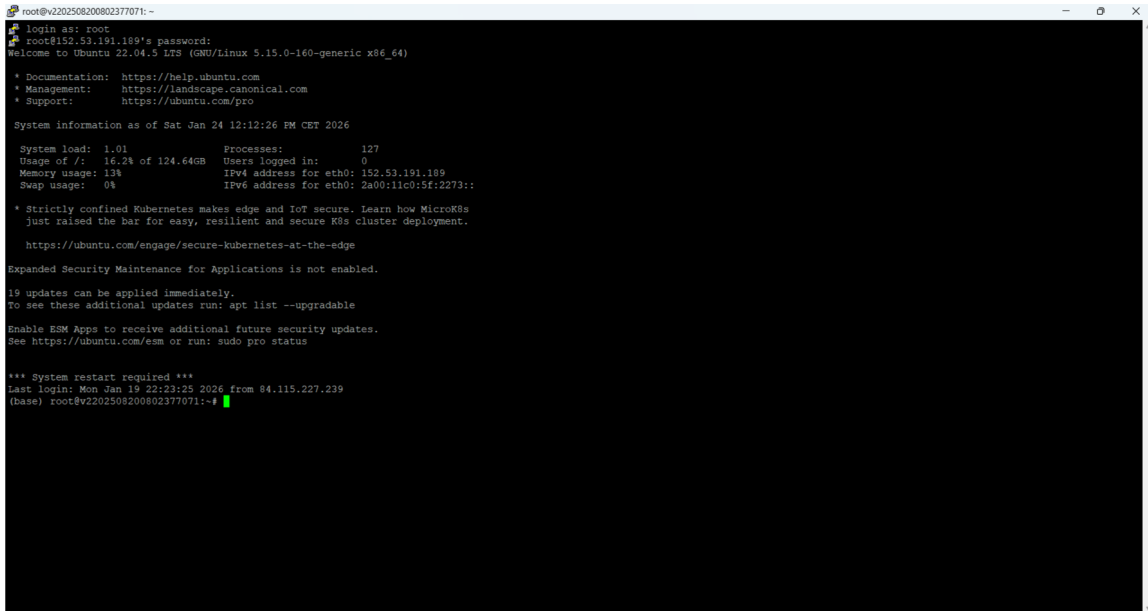
Both strategies were deployed as fully automated trading systems on a Linux server using Python and the `ib_insync` API. The implementation enforces trading hours, incorporates automated reconnection logic, and ensures mandatory disconnection outside active sessions.

8 Conclusion

This study demonstrates that strong In-Sample performance does not guarantee Out-of-Sample success in non-stationary financial markets. While the Momentum strategy performed exceptionally well during training, a subsequent regime shift resulted in significant losses. The adaptive strategy extension improved robustness by dynamically adjusting to evolving market conditions, illustrating a realistic and complete quantitative asset management workflow.

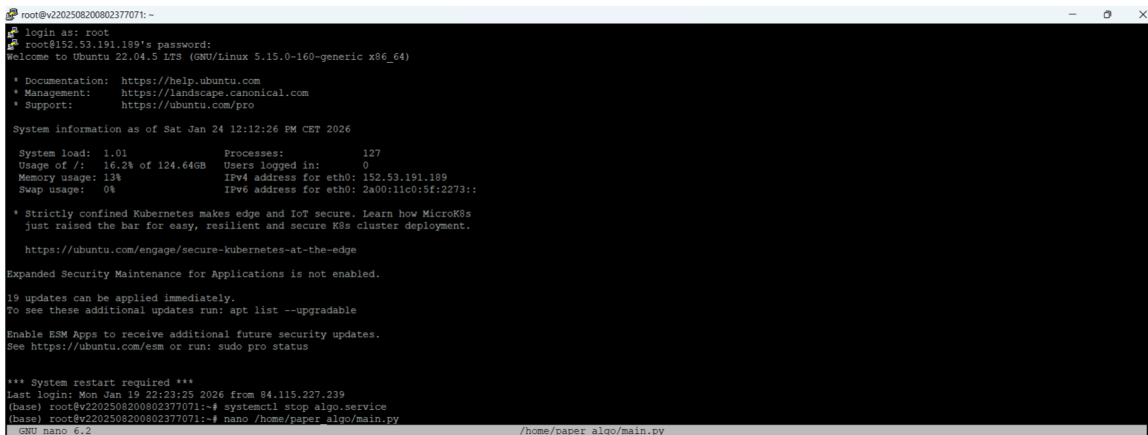
A Appendix A: Server Deployment – Base Momentum Strategy

This appendix documents the deployment of the base Momentum trading strategy on a remote Linux server. The screenshots demonstrate the full operational workflow, including server login, script execution, service configuration, and runtime verification.



```
root@v2202508200802377071: ~  
login as: root  
root@152.53.191.189's password:  
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-160-generic x86_64)  
  
 * Documentation:  https://help.ubuntu.com  
 * Management:    https://landscape.canonical.com  
 * Support:       https://ubuntu.com/pro  
  
System information as of Sat Jan 24 12:12:26 PM CET 2026  
  
System load: 1.01          Processes:      127  
Usage of /:  16.2% of 124.64GB  Users logged in:  0  
Memory usage: 13%          IPv4 address for eth0: 152.53.191.189  
Swap usage:  0%            IPv6 address for eth0: 2a00:11c0:5f:2273::  
  
 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s  
   just raised the bar for easy, resilient and secure K8s cluster deployment.  
  
   https://ubuntu.com/engage/secure-kubernetes-at-the-edge  
  
Expanded Security Maintenance for Applications is not enabled.  
  
19 updates can be applied immediately.  
To see these additional updates run: apt list --upgradable  
  
Enable ESM Apps to receive additional future security updates.  
See https://ubuntu.com/esm or run: sudo pro status  
  
*** System restart required ***  
Last login: Mon Jan 19 22:23:25 2026 from 84.115.227.239  
(base) root@v2202508200802377071:~
```

Figure 6: Secure login to the Ubuntu server hosting the trading algorithm.



```
root@v2202508200802377071: ~  
login as: root  
root@152.53.191.189's password:  
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-160-generic x86_64)  
  
 * Documentation:  https://help.ubuntu.com  
 * Management:    https://landscape.canonical.com  
 * Support:       https://ubuntu.com/pro  
  
System information as of Sat Jan 24 12:12:26 PM CET 2026  
  
System load: 1.01          Processes:      127  
Usage of /:  16.2% of 124.64GB  Users logged in:  0  
Memory usage: 13%          IPv4 address for eth0: 152.53.191.189  
Swap usage:  0%            IPv6 address for eth0: 2a00:11c0:5f:2273::  
  
 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s  
   just raised the bar for easy, resilient and secure K8s cluster deployment.  
  
   https://ubuntu.com/engage/secure-kubernetes-at-the-edge  
  
Expanded Security Maintenance for Applications is not enabled.  
  
19 updates can be applied immediately.  
To see these additional updates run: apt list --upgradable  
  
Enable ESM Apps to receive additional future security updates.  
See https://ubuntu.com/esm or run: sudo pro status  
  
*** System restart required ***  
Last login: Mon Jan 19 22:23:25 2026 from 84.115.227.239  
(base) root@v2202508200802377071:~# systemctl stop algo.service  
(base) root@v2202508200802377071:~# nano /home/paper_algo/main.py  
GNU nano 6.2 /home/paper_algo/main.py
```

Figure 7: Opening and editing the main trading script (main.py).

```

root@v2202508200802377071: ~
GNU nano 6.2 /home/paper_algo/main.py
from ib_async import *
import asyncio
import time

# --- CONFIGURATION ---
ib = IB()

# --- ROBUST CONNECTION LOOP ---
# This loop prevents the bot from crashing if Gateway is missing.
# It will keep the service "Green" (Active) while waiting.
print("🚀 Bot starting...")
while not ib.isConnected():
    try:
        print("🔄 Connecting to IB Gateway (Port 4002)...")
        ib.connect('127.0.0.1', 4002, clientId=99)
        print("✅ Connected Successfully!")
    except Exception as e:
        print(f"❌ Connection failed. Retrying in 10 seconds...")
        time.sleep(10)

contract = Forex('EURUSD')

# --- STRATEGY: MOMENTUM ---
def on_bar_update(bars, hasNewBar):
    if hasNewBar:
        if len(bars) < 11: return

        # Logic
        current_price = bars[-1].close
        past_price = bars[-11].close
        momentum = current_price - past_price

        # Positions
        positions = ib.positions()
        current_pos = sum(p.position for p in positions if p.contract.symbol == 'EUR')

        # Trading
        if momentum > 0 and current_pos <= 0:
            ib.placeOrder(contract, MarketOrder('BUY', 20000))
            print(f"📈 Momentum UP: BUY")
        elif momentum < 0 and current_pos >= 0:
            ib.placeOrder(contract, MarketOrder('SELL', 20000))
            print(f"📉 Momentum DOWN: SELL")

# --- EXECUTION ---

```

Figure 8: Implementation of the Momentum trading logic and order execution rules.

```

root@v2202508200802377071: ~
* Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
  just raised the bar for easy, resilient and secure K8s cluster deployment.

https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance for Applications is not enabled.

19 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

*** System restart required ***
Last login: Mon Jan 19 22:23:25 2026 from 84.115.227.239
(base) root@v2202508200802377071:~# systemctl stop algo.service
(base) root@v2202508200802377071:~# nano /home/paper_algo/main.py
(base) root@v2202508200802377071:~# ... connection code ...

# MOMENTUM LOGIC
current_price = bars[-1].close
past_price = bars[-11].close
momentum = current_price - past_price
# ... buying/selling logic ...
-bash: current_price: command not found
-bash: past_price: command not found
-bash: momentum: command not found
(base) root@v2202508200802377071:~# nano /home/paper_algo/main.py
(base) root@v2202508200802377071:~# systemctl start algo.service
(base) root@v2202508200802377071:~# systemctl status algo.service
● algo.service - EURUSD Trading Bot
   Loaded: loaded (/etc/systemd/system/algo.service; disabled; vendor preset: enabled)
   Active: active (running) since Sat 2026-01-24 12:25:50 CET; 22s ago
     Main PID: 2906219 (python)
       Tasks: 1 (limit: 4552)
        Memory: 40.7M
          CPU: 283ms
     CGroup: /system.slice/algo.service
            └─2906219 /root/miniconda3/envs/arima/bin/python main.py

Jan 24 12:25:50 v2202508200802377071 systemd[1]: Started EURUSD Trading Bot.
Jan 24 12:25:50 v2202508200802377071 python[2906219]: API connection failed: ConnectionRefusedError(111, "Connect call failed ('127.0.0.1', 4002)")
Jan 24 12:25:50 v2202508200802377071 python[2906219]: Make sure API port on TWS/IBG is open
Jan 24 12:26:00 v2202508200802377071 python[2906219]: API connection failed: ConnectionRefusedError(111, "Connect call failed ('127.0.0.1', 4002)")
Jan 24 12:26:00 v2202508200802377071 python[2906219]: Make sure API port on TWS/IBG is open
Jan 24 12:26:10 v2202508200802377071 python[2906219]: API connection failed: ConnectionRefusedError(111, "Connect call failed ('127.0.0.1', 4002)")
Jan 24 12:26:10 v2202508200802377071 python[2906219]: Make sure API port on TWS/IBG is open
(base) root@v2202508200802377071:~#

```

Figure 9: Systemd service configuration for automatic startup and monitoring of the Momentum bot.

B Appendix B: Server Deployment – Adaptive Strategy Extension

This appendix documents the deployment of the **Adaptive Strategy Extension**, which dynamically switches between Momentum and Mean Reversion regimes based on rolling Sharpe Ratio performance. The screenshots illustrate the replacement of the execution file, logic enhancements, and service restart verification.

```
root@v2202508200802377071: ~
login as: root
root@152.53.191.189's password:
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-160-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Sun Jan 25 03:50:45 PM CET 2026

System load: 1.08      Processes:    125
Usage of /: 16.2% of 124.64GB Users logged in: 0
Memory usage: 13%      IPv4 address for eth0: 152.53.191.189
Swap usage: 0%         IPv6 address for eth0: 2a00:11c0:5f:2273::

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance for Applications is not enabled.

25 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

*** System restart required ***
Last login: Sat Jan 24 12:12:27 2026 from 62.218.53.194
(base) root@v2202508200802377071:~# nano /home/paper_algo/main_adaptive.py
```

Figure 10: Uploading and opening the adaptive trading script (main_adaptive.py).

```
GNU nano 6.2 /home/paper_algo/main_adaptive.py
regime = "MEAN REV"
# MR Logic: Buy if Price < Average (Oversold), Sell if Price > Average (Overbought)
# Calculate 50-min Moving Average
ma_50 = np.mean(closes[-50:])

# MR Rules
if current_price < ma_50 and current_pos <= 0:
    ib.placeOrder(contract, MarketOrder('BUY', 20000))
    print(f"[{regime}] Sharpe (rolling_sharpe:.2f) < 0 | Price < MA | BUY (Dip)")
elif current_price > ma_50 and current_pos >= 0:
    ib.placeOrder(contract, MarketOrder('SELL', 20000))
    print(f"[{regime}] Sharpe (rolling_sharpe:.2f) < 0 | Price > MA | SELL (Peak)")

# --- EXECUTION ---
# Request 2 days of data (approx 3000 S * X7 Mo, use Duration String)
# We ask for '2 D' (2 Days) to fill the history immediately for the calculation
bars = ib.reqHistoricalData(contract, endTime='', durationStr='2 D',
    barSizeSetting='1 min', whatToShow='MIDPOINT', useRTH=True,
    keepUpToDate=True)

bars.updateEvent += on_bar_update

ib.run()
```

Figure 11: Mean Reversion logic implementation using a rolling moving average.

```
root@v2202508200802377071: ~
login as: root
root@152.53.191.189's password:
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-160-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Sun Jan 25 03:50:45 PM CET 2026

System load: 1.08      Processes:    125
Usage of /: 16.2% of 124.64GB Users logged in: 0
Memory usage: 13%      IPv4 address for eth0: 152.53.191.189
Swap usage: 0%         IPv6 address for eth0: 2a00:11c0:5f:2273::

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

https://ubuntu.com/engage/secure-kubernetes-at-the-edge

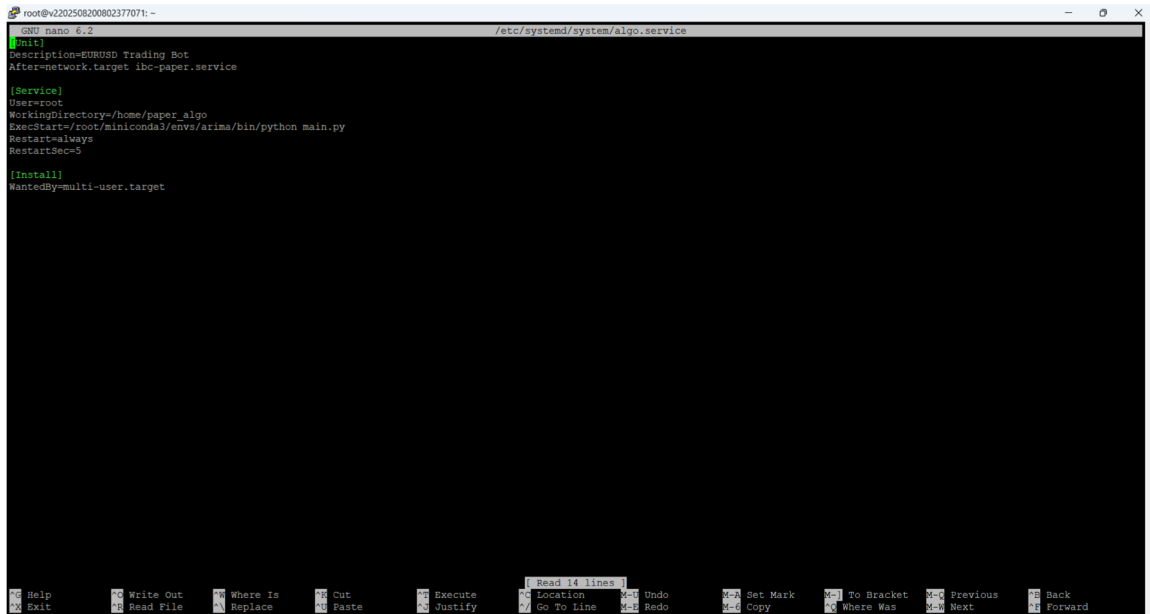
Expanded Security Maintenance for Applications is not enabled.

25 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

*** System restart required ***
Last login: Sat Jan 24 12:12:27 2026 from 62.218.53.194
(base) root@v2202508200802377071:~# nano /home/paper_algo/main_adaptive.py
(base) root@v2202508200802377071:~# nano /etc/systemd/system/algos.service
```

Figure 12: Execution rules for Momentum and Mean Reversion regimes based on rolling Sharpe Ratio.

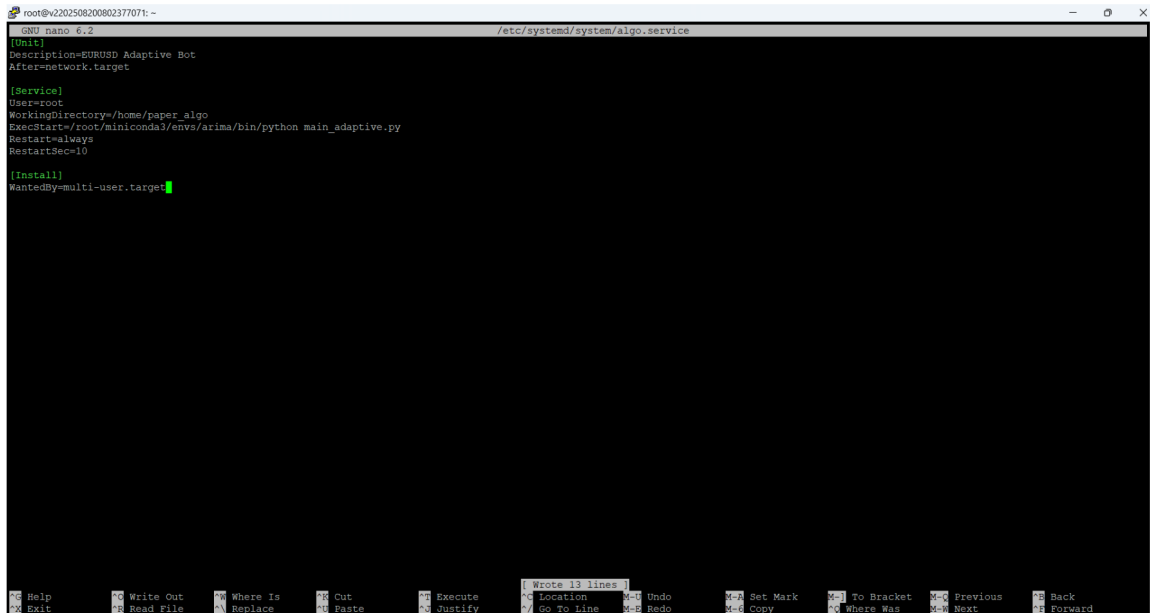


```
root@v2202508200802377071: ~  
GNU nano 6.2 /etc/systemd/system/algo.service  
[Unit]  
Description=EURUSD Trading Bot  
After=network.target ibc-paper.service  
  
[Service]  
User=root  
WorkingDirectory=/home/paper_algo  
ExecStart=/root/miniconda3/envs/arima/bin/python main.py  
Restart=always  
RestartSec=5  
  
[Install]  
WantedBy=multi-user.target  
  
Help Write Out Where Is Cut Execute Read 14 lines Location Undo W-A Set Mark W-B To Bracket W-C Previous B-B Back  
Exit Read File Replace Paste Justify Go To Line Redo M-A Copy M-B Where Was M-W Next M-D Forward
```

Figure 13: Historical data request and live bar update configuration for adaptive trading.

Replacement of execution file on the server

The original execution file (`main.py`) was replaced with the adaptive version (`main_adaptive.py`) to enable regime-aware trading behavior.



```
root@v2202508200802377071: ~  
GNU nano 6.2 /etc/systemd/system/algo.service  
[Unit]  
Description=EURUSD Adaptive Bot  
After=network.target  
  
[Service]  
User=root  
WorkingDirectory=/home/paper_algo  
ExecStart=/root/miniconda3/envs/arima/bin/python main_adaptive.py  
Restart=always  
RestartSec=10  
  
[Install]  
WantedBy=multi-user.target  
  
Help Write Out Where Is Cut Execute Wrote 13 lines Location Undo W-A Set Mark W-B To Bracket W-C Previous B-B Back  
Exit Read File Replace Paste Justify Go To Line Redo M-A Copy M-B Where Was M-W Next M-D Forward
```

Figure 14: Updating the systemd service file to point to the adaptive trading script.

```
root@v2202508200802377071: ~
GNU nano 6.2 /home/paper_algo/main_adaptive.py
# --- TRADING RULES ---
if sharpe > 0:
    # MOMENTUM MODE
    if momentum > 0 and current_pos <= 0:
        ib.placeOrder(contract, MarketOrder("BUY", 20000))
        print(f"MOMENTUM BUY (Sharpe {sharpe:.2f})")
    elif momentum < 0 and current_pos >= 0:
        ib.placeOrder(contract, MarketOrder("SELL", 20000))
        print(f"MOMENTUM SELL (Sharpe {sharpe:.2f})")
    else:
        # MEAN REVERSION MODE
        if bars[-1].close < ma_50 and current_pos <= 0:
            ib.placeOrder(contract, MarketOrder("BUY", 20000))
            print(f"MEAN REV BUY (Sharpe {sharpe:.2f})")
        elif bars[-1].close > ma_50 and current_pos >= 0:
            ib.placeOrder(contract, MarketOrder("SELL", 20000))
            print(f"MEAN REV SELL (Sharpe {sharpe:.2f})")

# --- EXECUTION ---
bars = ib.reqHistoricalData(contract, '', '1 D', '1 min', 'MIDPOINT', 1, 1, False, [])
bars.updateEvent += on_bar_update

ib.run()
```

Figure 15: Restarting the adaptive trading service on the Linux server.

```
(base) root@v2202508200802377071:~# nano /home/paper_algo/main_adaptive.py
(base) root@v2202508200802377071:~# systemctl restart algo.service
(base) root@v2202508200802377071:~# systemctl status algo.service
● algo.service - EURUSD Adaptive Bot
   Loaded: loaded (/etc/systemd/system/algo.service; disabled; vendor preset: enabled)
   Active: active (running) since Sun 2026-01-25 17:01:24 CET; 18s ago
     Main PID: 2957721 (python)
       Tasks: 1 (limit: 4552)
      Memory: 31.7M
         CPU: 22ms
    CGroup: /system.slice/algo.service
            └─2957721 /root/miniconda3/envs/arima/bin/python main_adaptive.py

Jan 25 17:01:24 v2202508200802377071 systemd[1]: Stopped EURUSD Adaptive Bot.
Jan 25 17:01:24 v2202508200802377071 systemd[1]: Started EURUSD Adaptive Bot.
Jan 25 17:01:24 v2202508200802377071 python[2957721]: API connection failed: ConnectionRefusedError(111, "Connect call failed ('127.0.0.1', 4002)")
Jan 25 17:01:24 v2202508200802377071 python[2957721]: Make sure API port on TWS/IBG is open
Jan 25 17:01:29 v2202508200802377071 python[2957721]: API connection failed: ConnectionRefusedError(111, "Connect call failed ('127.0.0.1', 4002)")
Jan 25 17:01:29 v2202508200802377071 python[2957721]: Make sure API port on TWS/IBG is open
Jan 25 17:01:34 v2202508200802377071 python[2957721]: API connection failed: ConnectionRefusedError(111, "Connect call failed ('127.0.0.1', 4002)")
Jan 25 17:01:34 v2202508200802377071 python[2957721]: Make sure API port on TWS/IBG is open
Jan 25 17:01:39 v2202508200802377071 python[2957721]: API connection failed: ConnectionRefusedError(111, "Connect call failed ('127.0.0.1', 4002)")
Jan 25 17:01:39 v2202508200802377071 python[2957721]: Make sure API port on TWS/IBG is open
(base) root@v2202508200802377071:~#
```

Figure 16: Service status confirmation showing the adaptive trading bot running successfully.